CAP 931 · AI PROMPT ENGINEERING CAPSTONE

Sales Account Research Assistant

A Streamlit agent that turns a company’s website into a one-page sales brief, built with a chained GPT prompt pipeline

Richard Watson Jr. Per Scholas · [Presentation date]

THE PROBLEM

A new sales rep has 10 minutes to prepare for a first meeting

Who is this company?

What it does, how big it is, and what it has been working on lately

Who am I meeting?

The leaders, their titles, and why each one might care about the product

What do I say?

Competitor mentions, likely needs, and talking points, with sources

The capstone: build an AI agent that collects this from public web data and returns a one-page brief.

THE TEST CASE

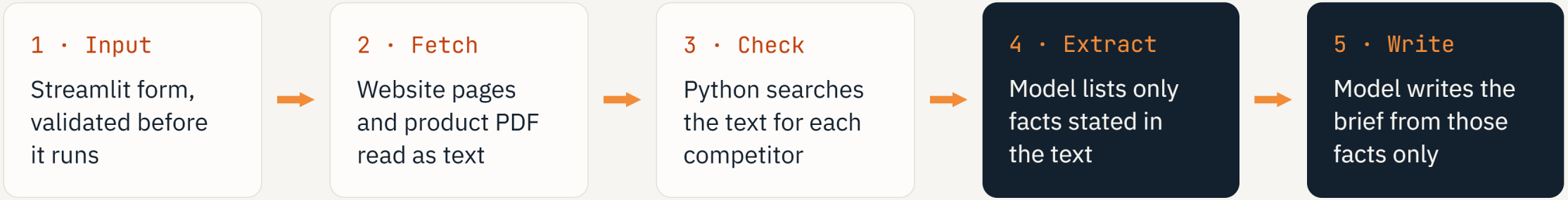
One standard scenario, used for every experiment

Field	Value
Product	GreatShield EDR (fictional endpoint security product)
Prospect	Municipal Credit Union, nymcu.org
Extra pages	About, Leadership Team, MCU in the Community
Competitors	CrowdStrike, SentinelOne
Meeting with	Kelly Gerber, Chief Risk Officer
Product sheet	1-page PDF I wrote for GreatShield EDR

Same inputs every run, so a change in output comes from the prompt, not the inputs.

HOW IT WORKS

From a form to a one-page brief in five steps



Light boxes are plain Python. Dark boxes are calls to the model. Python adds the list of source links at the end.

TOOLS AND MODEL

A small, free stack built around one GPT model

Job	Tool
Interface	Streamlit
Language	Python with uv
Web pages	requests, BeautifulSoup
Product PDF	pypdf
Model	gpt-oss-120b on Groq
Version control	Git

Why this model

- An OpenAI GPT model, as the assignment asks
- Free to use through Groq, no paid API key
- Fast enough for two calls per brief
- Large context window fits several web pages

PROMPT EXPERIMENTS

Seven prompt versions, one change at a time

Ver.	What changed	What happened
v1	Role, task, context, output rules	Clean format; invented stats
v2	+ “Use only the information provided”	Fakes gone; almost nothing known
v3	+ real website text	Grounded facts; 3 targeted unknowns
v4	+ competitor check, Python sources	Accurate competitor section
v5	+ leadership pages, rubric headings	Real leaders; invented features
v6	Chaining: extract, then write	Invented features gone
v7	+ product PDF, tighter rules	Real features; needs still biased

V1 → V3 · HALLUCINATION

A prompt can stop a model from inventing facts. It can’t give it facts.

v1 · No data

Made up a case study: “a similar credit union reduced ransomware risk by 92%.” The product doesn’t exist.

v2 · Rule only

“Write ‘Unknown – verify’ instead of guessing.” Fakes gone, but it didn’t even know MCU is a credit union.

v3 · Real data

Fed the homepage text. Real products and initiatives; only three specific unknowns left.

V6 · PROMPT CHAINING

Splitting one big prompt into two small ones

Step 1 · Extract

Raw web text in. A list of stated facts out, under five headings. No interpreting.

Step 2 · Write

Facts and form inputs in. The brief out. It never sees the messy web text.

Issue	One prompt	Chained
Invented features	Yes	Gone
Invented pricing	Yes	Gone
Leader roles	Stated as fact	Labeled “likely”
Leaders named	5	7

Trade-off: two model calls, so slower, but more accurate and easier to check.

DEBUGGING STORY

The missing leadership names: test the cause first

Symptom

The brief said no leaders were found, though the leadership page lists them.

Test

```
in downloaded text: True
in prompt text: False
```

Downloaded, then cut off. The 3,000-character slice held only the site menu.

Fix

Strip nav, header and footer before keeping text. One function, so every page was fixed at once.

1 leader → 7

DESIGN PRINCIPLE

Let code do what code does reliably

Python handles

- Checking required fields and URL format
- Downloading pages and stripping menus
- Searching for each competitor name
- Reading the product PDF
- Listing the source links
- Skipping a page that fails to load

The model handles

- Pulling facts out of messy page text
- Choosing which leaders matter
- Writing needs and talking points
- Tailoring the pitch to the person

THE RESULT

A one-page brief for the MCU meeting

[Screenshot of the finished one-pager goes here]

What it found

- Member-owned, 630,000+ members
- Recent news, e.g. VaultRight selection, new Chelsea branch
- Executive team with titles
- No CrowdStrike or SentinelOne mentions
- Every source page listed

Leader names verified: [__/8]

HONEST LIMITATIONS

What the agent still gets wrong

Limitation	Plan or mitigation
Customer needs borrow product features	Separate needs step without the product sheet
“Not on the website” ≠ “not a customer”	Rule added; rep must still ask
Only pages the rep pastes are searched	Add news search and job postings
No link to each news article yet	Capture article URLs from the page
Output varies run to run	Lower temperature; human review
JavaScript-built pages may load empty	Use a headless browser if needed

WHAT I LEARNED

Four prompt engineering lessons

1 · Ground it in real data

Rules reduce invention; only data adds facts.

2 · Structure beats more rules

Control what the model sees, not just what it’s told.

3 · Give code the exact jobs

Searches, links and checks don’t need a model.

4 · Test, don’t guess

One variable per experiment, one fixed test case.

Questions?

Live demo and code available on request

Demo: cap931-sales-agent-rwjr.streamlit.app

Code: github.com/rwnycvet/cap931-sales-agent

Richard Watson Jr. · Per Scholas